

SUPSI

Branch and Bound as PTAS

Studente/i

Tiziano Zamaroni

Sergio Aquilini

Relatore

Monaldo Mastrolilli

Correlatore

-

Committente

SUPSI

Corso di laurea

C-P6051P - Seminari di Progetto

Modulo

**M-P6050P - Progetto di
Semestre**

Anno

2025

Data

29 novembre 2025

Indice

1	Introduzione	1
2	Motivazione e Contesto	3
2.1	Motivazione	3
2.2	Contesto	3
3	Algoritmi	5
3.1	Approssimazione soluzione	5
3.2	Branch-and-Bound	5
3.3	Greedy	5
3.4	Profiling	5
4	Requisiti	7
4.1	Requisiti funzionali	7
4.2	Requisiti non funzionali	8
5	Tecnologie utilizzate	9
5.1	Implementazione algoritmi	9
5.2	Analisi dei dati	9
6	Implementazione	11
6.1	Architettura e struttura del codice	11
6.2	Branch-and-Bound	11
6.3	Greedy	11
6.4	Profiling	11
6.5	Generazione istanze	11
6.6	Analisi dei dati	11
7	Analisi dei risultati	13
8	Metodologia di lavoro	15

9	Conclusione	17
9.1	Risultati ottenuti	17
9.2	Possibili miglioramenti futuri	17

Elenco delle figure

Elenco delle tabelle

1 Introduzione

- Problemi NP-Hard
- Branch-and-Bound Algorithms as Polynomial-time Approximation Schemes
- Unrelated parallel machine scheduling problem
- Uniform/Identical parallel machine scheduling problem (identical caso particolare di uniform)
- Profiling come ulteriore ottimizzazione di identical machine scheduling
- Dimostrazione pratica della validità dell'approccio con profiling

2 Motivazione e Contesto

2.1 Motivazione

2.2 Contesto

3 Algoritmi

3.1 Approssimazione soluzione

Differenze del valore di epsilon tra i vari problemi

3.2 Branch-and-Bound

3.3 Greedy

3.4 Profiling

4 Requisiti

Il lavoro di semestre si inserisce nel contesto del paper *Branch-and-Bound Algorithms as Polynomial-time Approximation Scheme* [1], scritto dal nostro relatore Prof. Monaldo Mastroilli insieme a István Encz Koppányi ed Eleonora Vercesi. Gli autori forniscono un'implementazione Python del Branch-and-Bound per il caso delle *unrelated machines*, comprensiva di euristiche e gestione dei bound. Il nostro compito consiste nel comprendere a fondo tale implementazione, adattarla al caso delle *identical machines* e integrare la tecnica di *profiling*, applicabile poiché le identical machines rappresentano un caso particolare di uniform machines. L'obiettivo finale è verificare sperimentalmente quanto presentato nel paper, confrontando la versione base dell'algoritmo con quella dotata di profiling.

4.1 Requisiti funzionali

Il progetto richiede i seguenti requisiti funzionali:

RF1 – Solver per identical machines

Realizzare una variante dell'algoritmo Branch-and-Bound in grado di risolvere il problema di scheduling su identical machines, adattando la struttura e le funzioni dell'implementazione esistente per le unrelated machines.

RF2 – Integrazione del profiling

Implementare il meccanismo di profiling descritto nel paper, così da ottenere una seconda variante dell'algoritmo direttamente confrontabile con la versione base.

RF3 – Generazione ed esecuzione degli esperimenti

Eseguire entrambe le varianti dell'algoritmo e registrare i risultati, raccogliendo una quantità consona di dati.

RF4 – Raccolta dei dati

Memorizzare in formato strutturato informazioni quali nodi esplorati, tempo di esecuzione, delta rispetto alla soluzione ottima dell'istanza considerata e altre metriche rilevanti.

RF5 – Validazione sperimentale

Analizzare i dati raccolti, con l'aiuto di strumenti statistici e di visualizzazione, per verificare l'efficacia del profiling rispetto alla versione base dell'algoritmo.

4.2 Requisiti non funzionali

Non erano previsti requisiti non funzionali formali, ma il progetto implicava alcuni vincoli pratici:

Efficienza computazionale

L'implementazione e la scelta delle istanze dovevano permettere di generare e analizzare un numero significativo di dati entro i limiti temporali del lavoro di semestre.

Ripetibilità

La generazione delle istanze e l'esecuzione degli algoritmi dovevano essere riproducibili tramite seed controllati.

Mantenibilità e pulizia del codice

Anche trattandosi di un esperimento non destinato a evoluzione futura, il codice doveva essere scritto in modo chiaro e integrarsi coerentemente con la struttura esistente.

5 Tecnologie utilizzate

5.1 Implementazione algoritmi

L'implementazione degli algoritmi è stata sviluppata in *Python* a partire dal codice fornito dagli autori del paper [1], originariamente progettato per il caso delle *unrelated machines*. In tale versione, il lower bound dei nodi del Branch-and-Bound veniva calcolato tramite il solver *SCIP* [2], utilizzato attraverso la libreria *PySCIPOpt*. Questa scelta, appropriata per il problema generale, consente agli autori di ottenere bound frazionari mediante *binary search* e *linear relaxation*.

Nel nostro progetto, tuttavia, tale approccio si è rivelato eccessivamente oneroso e non strettamente necessario. Nel caso delle *identical machines*, infatti, la struttura del problema è significativamente più semplice e permette l'impiego di metodi più leggeri per il calcolo del lower bound. Inoltre, l'utilizzo di *SCIP* [2] presentava un ulteriore limite pratico: non è stato possibile forzare il solver a utilizzare sistematicamente il metodo *primal simplex*, condizione necessaria per garantire che il numero di variabili frazionarie nella soluzione LP rimanesse inferiore o uguale al numero di macchine. L'assenza di tale garanzia comprometteva l'affidabilità della fase di arrotondamento e introduceva un overhead computazionale non trascurabile.

Per queste ragioni, abbiamo sostituito integralmente l'utilizzo di *SCIP* [2] con un metodo di calcolo del lower bound basato su un criterio *greedy*, pienamente sufficiente per il caso delle *identical machines* e decisamente più efficiente dal punto di vista computazionale. Questa scelta ha permesso di ridurre drasticamente il tempo di elaborazione dei nodi del Branch-and-Bound, mantenendo comunque una qualità del bound adeguata agli scopi del progetto.

5.2 Analisi dei dati

L'analisi dei dati sperimentali è stata effettuata in *Python* utilizzando le principali librerie per la data science. *numpy* è stato impiegato per l'elaborazione numerica di base, *pandas* per l'organizzazione e l'aggregazione dei risultati raccolti durante gli esperimenti, e *matplotlib* per la generazione delle visualizzazioni utilizzate nel confronto tra la versione base

dell'algoritmo e quella dotata di profiling. I dati sono stati esportati in formato CSV e analizzati tramite notebook dedicati, permettendo di sintetizzare l'andamento delle performance e supportare quantitativamente la discussione dei risultati.

6 Implementazione

6.1 Architettura e struttura del codice

6.2 Branch-and-Bound

6.3 Greedy

6.4 Profiling

6.5 Generazione istanze

6.6 Analisi dei dati

7 Analisi dei risultati

- Overview istanze e dati raccolti
- Istanze relativamente piccole ma con diversi seed
- Difficoltà variabile a dipendenza del seed, a parità di dimensione
- Statistiche no-profiling vs. prune per tipo di istanza
- Evidenziare che profiling rimane percorribile quando la versione base non arriva a soluzione

8 Metodologia di lavoro

- Suddivisione dei compiti
- Pianificazione e Task
- Pair programming / brainstorming
- Weekly
- Raccolta dei dati

9 Conclusione

9.1 Risultati ottenuti

9.2 Possibili miglioramenti futuri

Elenco di possibili sviluppi futuri:

- abc
- def

Bibliografia

- [1] Koppányi István Encz, Monaldo Mastrolilli, and Eleonora Vercesi. Branch-and-bound algorithms as polynomial-time approximation scheme. *Mathematics of Operations Research*, 2025.
- [2] Scip optimization suite. <https://scipopt.org>.